ALGEBRAIC NUMBER THEORY / WOLFRAM LANGUAGE

# Decomposing Polynomial Roots

## into Low-Degree Sums and Products

Exact discovery, optimality certificates, and the distinction between two factors and arbitrary arity

A technical article for Vladimir Reshetnikov

6 September 2026

**Abstract**

Given an exact algebraic number, the objective is to express it as a flat sum or product whose operands have the smallest possible maximum absolute degree. Both degree-nine examples in the question admit the requested cubic decompositions, and degree three is globally minimal even when arbitrarily many complex algebraic operands are allowed.

For sums, normalized trace reduces the unrestricted problem to linear algebra in finitely many embedded subfields of a normal closure. For products of two factors, a power-and-norm reduction gives another finite criterion: test intersections of a subfield with a scalar multiple of another subfield. This avoids solving bilinear coefficient systems and does not assume that the original factors lie in the normal closure. A counterexample shows that this complete two-factor algorithm cannot be turned into an arbitrary-arity algorithm by recursively splitting into smaller degrees.

The accompanying Wolfram Language package implements exact certificates, bounded-height discovery, embedded-subfield enumeration, the sum algorithm, the two-factor algorithm, and a bounded arbitrary-arity product search. Proofs, explicit recovery formulas, error semantics, and reproducibility files accompany the implementation.

**Scope and verification.** The two supplied examples are solved with global optimality proofs. The sum algorithm is complete with exhaustive Galois-field data; the product algorithm is complete for *two* factors. The arbitrary-arity product search is bounded, not an unconditional terminating global minimizer. Forty independent exact Python/SymPy checks passed. Native Wolfram regression tests are supplied but were not run: the available Wolfram connector returned HTTP 404.

# Contents

# 1 The problem and the guarantees

For an algebraic number $a \in \overline{\mathbb{Q}} \subset \mathbb{C}$, write

$$\deg_{\mathbb{Q}}(a) = [\mathbb{Q}(a) : \mathbb{Q}].$$

All degrees in this article are *absolute* degrees over $\mathbb{Q}$. A degree-two polynomial with an algebraic coefficient does not by itself give an operand of absolute degree two. We measure the minimal polynomial, not merely the degree of the polynomial used in an input `Root` expression. The documented semantics of `Root`, `MinimalPolynomial`, and `RootReduce` supply the exact algebraic-number primitives used below [5, 6, 7].

Define

$$D_{+}(a) = \min_{a=b_1+\cdots+b_r} \max_i \deg_{\mathbb{Q}}(b_i), \tag{1}$$

$$D_{\times}(a) = \min_{a=b_1\cdots b_r} \max_i \deg_{\mathbb{Q}}(b_i), \tag{2}$$

where $r \geq 1$ is finite but is not specified in advance. The expressions are *flat*: a nested expression built from small relative extensions is a different objective. We also distinguish

$$D_{\times,2}(a) = \min_{a=bc} \max\{\deg_{\mathbb{Q}}(b), \deg_{\mathbb{Q}}(c)\}.$$

Rational operands are permitted. Consequently all three quantities are at most $\deg_{\mathbb{Q}}(a)$, using $a$ alone or $a \cdot 1$. For $a \in \mathbb{Q}$, including zero, their value is one. When discussing nontrivial products we assume $a \neq 0$, so every factor is nonzero.

The original question asks for several operands rather than exactly two [1]. This distinction is mathematically consequential, not just an implementation option. The guarantees established here are as follows.

| Task | Guarantee in this article |
|---|---|
| The two degree-nine examples | Exact requested branches, automatic bounded discovery, and proof that the global minimum is three. |
| Arbitrary-arity sums | A finite, complete minimization algorithm using a Galois ambient field and exhaustive embedded subfields. |
| Products of two factors | A finite, complete minimization algorithm, including factors outside the ambient field. |
| Arbitrary-arity products | Exact bounded search and global certificates when a lower bound is attained; a general positive-witness enumeration, but no claimed terminating negative decision procedure. |

The default domain allows complex operands. Both requested witnesses happen to be real, and their lower bound remains valid even against complex competitors. An additional requirement that every operand be real changes the general product decision problem; one must not simply discard complex output branches and retain a completeness claim.

# 2 The requested decompositions, with exact recovery formulas

Set

$$f(X) = X^3 + X + 1, \qquad u = \texttt{Root[1 + \# + \#\^3 \&, 1]},$$

$$g(X) = X^3 - X + 1, \qquad v = \texttt{Root[1 - \# + \#\^3 \&, 1]}.$$

Both cubics have exactly one real root. Their discriminants are $-31$ and $-23$, respectively. The target polynomials are

$$P(Z) = Z^9 + 2Z^7 - 3Z^6 + Z^5 - Z^4 + 3Z^3 - Z - 1, \tag{3}$$

$$S(Z) = Z^9 + 6Z^6 + 3Z^5 - 15Z^3 + 24Z^2 - 4Z + 8. \tag{4}$$

The independent exact computations in the archive verify that $P$ and $S$ are irreducible and each has exactly one real root. Thus the index-one `Root` objects in the question are precisely

$$p = uv, \qquad s = u + v.$$

The branch claim follows from real-root isolation, not from a numerical guess. For orientation only,

$$u \approx -0.682327803828019, \qquad v \approx -1.324717957244746.$$

Exact Wolfram certification is direct:

```
alphaP = Root[-1 - # + 3 #^3 - #^4 + #^5 - 3 #^6 +
   2 #^7 + #^9 &, 1];
alphaS = Root[8 - 4 # + 24 #^2 - 15 #^3 + 3 #^5 +
   6 #^6 + #^9 &, 1];
u = Root[1 + # + #^3 &, 1];
v = Root[1 - # + #^3 &, 1];

RootReduce[alphaP - u v]      (* exact value: 0 *)
RootReduce[alphaS - u - v]    (* exact value: 0 *)
```

The stated exact values are supported independently by the polynomial identities and isolation checks in the supplied verifier; they are not presented as a transcript of a native Wolfram session.

## 2.1 Recovering the cubic factors from the product

The cubic equations give $u^3 = -u - 1$ and $v^3 = v - 1$. Multiplying them yields

$$p^3 = (-u - 1)(v - 1) = -p + u - v + 1.$$

Therefore, with

$$\delta = p^3 + p - 1,$$

we have $u - v = \delta$. The relations $uv = p$ and $v = u - \delta$ imply $u^2 = \delta u + p$, hence

$$0 = u^3 + u + 1 = (\delta^2 + p + 1)u + \delta p + 1.$$

We obtain the compact recovery formulas

$$\boxed{u = -\frac{\delta p + 1}{\delta^2 + p + 1}, \qquad v = u - \delta, \qquad \delta = p^3 + p - 1.} \tag{5}$$

For the real target $p = uv > 0$, the denominator is positive. More generally its polynomial is relatively prime to $P$, so it is invertible in $\mathbb{Q}[Z]/(P)$.

Reducing these rational functions modulo $P$ gives especially simple power-basis representatives:

$$u = -\frac{p^7 + 2p^5 - 2p^4 + 1}{2}, \tag{6}$$

$$v = -\frac{p^7 + 2p^5 - 2p^4 + 2p^3 + 2p - 1}{2}. \tag{7}$$

Thus the two cubic fields are already visibly embedded in the degree-nine root field $\mathbb{Q}(p)$.

### 2.2 Recovering the cubic summands from the sum

Putting $v = s - u$ in $g(v) = 0$ and adding $f(u) = 0$ gives

$$Au^2 + Bu + C = 0, \quad A = 3s, \quad B = 2 - 3s^2, \quad C = s^3 - s + 2.$$

Elimination of $u^3$ against $u^3 + u + 1 = 0$ yields

$$(B^2 - AC + A^2)u + BC + A^2 = 0.$$

After simplification,

$$\boxed{u = \frac{3s^5 - 5s^3 - 3s^2 + 2s - 4}{6s^4 - 6s + 4}, \qquad v = s - u.} \tag{8}$$

The denominator is relatively prime to $S$, as checked exactly. Substituting these expressions into $f(u)$ and $g(v)$ and reducing modulo $S$ gives zero. Since the target and the recovery expressions are real, the recovered roots are the unique real roots of the cubics.

These formulas solve the two particular inputs without a search. The remaining sections explain how to discover such decompositions and which searches establish minimality.

## 3 A lower bound valid for any number of operands

Let $P^+(n)$ denote the largest prime factor of a positive integer $n > 1$, and put $P^+(1) = 1$.

**Theorem 3.1** (Prime-divisor obstruction)**.** *For every algebraic number $a$,*

$$D_+(a) \geq P^+(\deg_{\mathbb{Q}}(a)), \qquad D_\times(a) \geq P^+(\deg_{\mathbb{Q}}(a)).$$

*The same lower bound applies to any finite expression formed by field operations on operands of degree at most $d$.*

*Proof.* Suppose the operands have degrees $m_1, \ldots, m_r \leq d$. Let $L_i$ be the splitting field of the minimal polynomial of the $i$th operand, and let $L$ be their compositum. Restriction embeds

$$\mathrm{Gal}(L/\mathbb{Q}) \hookrightarrow \prod_{i=1}^{r} \mathrm{Gal}(L_i/\mathbb{Q}) \hookrightarrow \prod_{i=1}^{r} S_{m_i}.$$

Every prime divisor of $[L : \mathbb{Q}]$ is therefore at most $d$. The value $a$ lies in $L$, so $\deg_{\mathbb{Q}}(a)$ divides $[L : \mathbb{Q}]$. A prime divisor of $\deg_{\mathbb{Q}}(a)$ exceeding $d$ is impossible. □

For each requested input, $\deg_{\mathbb{Q}}(a) = 9$ and $P^+(9) = 3$. The cubic witnesses prove the matching upper bound. Consequently

$$\boxed{D_\times(p) = 3, \qquad D_+(s) = 3.} \tag{9}$$

This excludes not only two quadratic operands, but *every finite collection* of quadratic operands.

For *two* operands there is another useful bound:

$$\deg_{\mathbb{Q}}(a) \leq [\mathbb{Q}(b, c) : \mathbb{Q}] \leq \deg_{\mathbb{Q}}(b) \deg_{\mathbb{Q}}(c) \leq d^2.$$

Thus $d \geq \lceil \sqrt{\deg_{\mathbb{Q}}(a)} \rceil$ for a two-operand representation. This square-root bound must not be used for arbitrary arity: with $r$ operands the corresponding crude upper bound is $d^r$.

### 3.1 A stronger obstruction when Galois information is available

Let $K_a$ be the normal closure of $\mathbb{Q}(a)$, and write $\exp(G)$ for the exponent of a finite group. The same splitting-field argument gives

$$\exp \mathrm{Gal}(K_a/\mathbb{Q}) \mid \mathrm{lcm}(1, 2, \ldots, d) \tag{10}$$

as a necessary condition for either kind of decomposition. Indeed, the exponent of a subgroup of the above product divides the indicated least common multiple, and the Galois group of $K_a$ is a quotient of that subgroup. The standard field and Galois correspondence facts used here are reviewed in Milne's notes [4].

For a primitive fifth root of unity $\zeta_5$, the normal closure is $\mathbb{Q}(\zeta_5)$ and its Galois group is cyclic of order four. Neither $d = 2$ nor $d = 3$ satisfies (10). Hence

$$D_+(\zeta_5) = D_\times(\zeta_5) = 4,$$

although the prime-divisor bound alone gives only two. The package automatically uses the elementary prime-divisor bound; it does not pretend to infer a Galois group from a heuristic.

## 4 Resultants: forward composition and bounded discovery

Let $f, g \in \mathbb{Q}[X]$ be monic of degrees $m, n$, with roots $u_i, v_j$. Their sum and product composition polynomials are

$$R_+(Z) = \mathrm{Res}_X(f(X), g(Z - X)) = \prod_{i,j}(Z - u_i - v_j), \tag{11}$$

$$R_\times(Z) = \mathrm{Res}_X(f(X), X^n g(Z/X)) = \prod_{i,j}(Z - u_i v_j). \tag{12}$$

For the second formula, write $g(Y) = \sum_{j=0}^{n} b_j Y^j$ and form the honest polynomial

$$X^n g(Z/X) = \sum_{j=0}^{n} b_j Z^j X^{n-j}.$$

This avoids evaluating a rational expression at a potentially zero root. The resultant identities follow by evaluating the second polynomial at each root of $f$; `Resultant` implements this elimination operation [8].

If a selected value $a$ is one of these sums or products, then its minimal polynomial divides the corresponding resultant. Equality is justified only when the minimal polynomial has the full degree $mn$, after monic normalization. Requiring equality in all cases would miss decompositions with coinciding pair values or a smaller compositum.

For the two cubics in Section 2, formulas (11) and (12) give exactly $S$ and $P$. The package implements the denominator-cleared product formula and normalizes nonmonic input polynomials before forming a resultant.

### 4.1 The companion-matrix interpretation

If $C_f, C_g$ are companion matrices, then

$$C_f \otimes I_n + I_m \otimes C_g \quad \text{and} \quad C_f \otimes C_g$$

have characteristic polynomials $R_+$ and $R_\times$. Over a splitting field, the companion matrices are diagonalizable because characteristic zero makes the irreducible polynomials separable; the claimed eigenvalues follow at once. The original Mathematica discussion contains companion-matrix and resultant approaches for prescribed degree patterns, due to yarchik and Daniel Lichtblau [1]. These are useful discovery mechanisms, but a chosen degree pattern and a solved coefficient system are not by themselves global minimality certificates.

### 4.2 Why unconstrained coefficient solving is not a full solution

Writing unknown coefficients for $f$ and $g$ and equating a resultant to the target creates polynomial equations in the coefficients. Several traps remain. A generic algebraic solution of these equations may give algebraic coefficients rather than rational ones. Integer or rational solvability is an additional requirement. Product scalings cannot in general normalize an arbitrary constant coefficient to one by a rational scaling: that would require an appropriate rational power. Finally, the equations identify pair values collectively, not the selected `Root` branch.

A finite coefficient box avoids the first two ambiguities. Enumerate primitive integer polynomials

$$a_m X^m + \cdots + a_0, \quad 1 \le a_m \le H, \quad |a_j| \le H, \quad \gcd(a_0, \ldots, a_m) = 1,$$

and retain irreducible polynomials over $\mathbb{Q}$. Allowing $a_m > 1$ matters: algebraic factors need not be algebraic integers. This enumerates all rational minimal polynomials as $H$ increases, without pretending that one fixed height bound is exhaustive.

### 4.3 A usable bounded search

After loading the package, use

```
Get["/absolute/path/to/code/RootDecomposition.wl"];

rP = FindRootPairByHeight[alphaP, "Product",
  "MaximumDegree" -> 3, "CoefficientHeight" -> 1,
  "TimeConstraint" -> 120];
rS = FindRootPairByHeight[alphaS, "Sum",
  "MaximumDegree" -> 3, "CoefficientHeight" -> 1,
  "TimeConstraint" -> 120];
```

For each degree bound, the routine enumerates polynomial pairs, rejects pairs with $mn < \deg_{\mathbb{Q}}(a)$, tests divisibility by the target minimal polynomial, and then compares all exact branch pairs by `RootReduce`. A successful answer contains `"Terms"`, `"Degrees"`, `"MaximumDegree"`, and `"GlobalMinimumProved"`.

The independent Python implementation found $X^3 - X + 1$ and $X^3 + X + 1$ for the sum. For the product, its enumeration order first found $X^3 - X - 1$ and $X^3 + X - 1$, whose real roots are $-v, -u$. Their product is the same target. The original negative pair $u, v$ is separately certified. Nonuniqueness of a valid witness is expected.

A return value `"NotFoundInBounds"` means exactly that the finite box was exhausted. It does not mean that a decomposition does not exist. Likewise, `Failure["TimeLimit", ...]` is a resource event, not a mathematical answer.

## 5 Complete minimization for arbitrary-arity sums

The essential simplification is to work in a finite Galois extension $K/\mathbb{Q}$ containing the target. The normal closure of $\mathbb{Q}(a)$ is always a valid choice. Altamirano's paper describes the trace-descent approach to sums [2]; the precise degree-bound formulation below also explains why an unspecified number of summands does not force an unbounded search over arities.

**Lemma 5.1** (Intersection and scalar extension)**.** *If $K/\mathbb{Q}$ is finite Galois and $b$ is algebraic, put $F = K \cap \mathbb{Q}(b)$. Then*

$$[K(b) : K] = [\mathbb{Q}(b) : F],$$

*and the minimal polynomial of $b$ over $K$ equals its minimal polynomial over $F$.*

*Proof.* Choose a common finite Galois overfield $M$, and put $G = \mathrm{Gal}(M/\mathbb{Q})$, $H = \mathrm{Gal}(M/K)$, and $J = \mathrm{Gal}(M/\mathbb{Q}(b))$. Normality of $K/\mathbb{Q}$ makes $H$ normal in $G$. The compositum corresponds to $H \cap J$, and the intersection corresponds to $HJ$. Hence

$$[K(b) : K] = [H : H \cap J] = [HJ : J] = [\mathbb{Q}(b) : F].$$

The minimal polynomial over $K$ divides the monic polynomial over $F$. Their equal degrees make them identical. □

**Theorem 5.2** (Trace descent without increasing operand degree)**.** *If $a \in K$ and $a = b_1 + \cdots + b_r$, there are $c_i \in K$ such that*

$$a = c_1 + \cdots + c_r, \qquad \deg_{\mathbb{Q}}(c_i) \le \deg_{\mathbb{Q}}(b_i).$$

*More precisely, $c_i$ can be chosen in $K \cap \mathbb{Q}(b_i)$.*

*Proof.* Take a finite extension $M/K$ containing all $b_i$, and set

$$c_i = \frac{\mathrm{Tr}_{M/K}(b_i)}{[M : K]}.$$

Linearity of trace and $a \in K$ give $\sum_i c_i = a$. If $t_i = [K(b_i) : K]$, transitivity of trace gives

$$c_i = \frac{\mathrm{Tr}_{K(b_i)/K}(b_i)}{t_i}.$$

By Lemma 5.1, the coefficients of the relevant minimal polynomial lie in $F_i = K \cap \mathbb{Q}(b_i)$, so its trace does too. Thus $c_i \in F_i$, and $[F_i : \mathbb{Q}] \le \deg_{\mathbb{Q}}(b_i)$ proves the degree assertion. □

Let $\mathcal{F}_d(K)$ be the finite set of all *embedded* intermediate fields $F \subseteq K$ with $[F : \mathbb{Q}] \le d$, and define

$$V_d(K) = \sum_{F \in \mathcal{F}_d(K)} F \qquad \text{as a rational vector subspace of } K. \tag{13}$$

**Corollary 5.3** (Finite criterion for arbitrary-arity sums)**.** *For $a \in K$,*

$$D_+(a) \le d \quad \Longleftrightarrow \quad a \in V_d(K).$$

*Proof.* Trace descent proves the forward implication. Conversely, membership in the sum of the fields gives a finite sum of elements, each lying in a field of degree at most $d$. □

## 5.1 The linear system and witness reconstruction

Choose a rational basis $e_1, \ldots, e_N$ of $K$, and a rational basis of each eligible subfield. Put the coordinate vectors of these subfield basis elements in the columns of a matrix $B_d$. If $\mathbf{a}$ is the coordinate vector of $a$, solve

$$B_d \mathbf{c} = \mathbf{a} \qquad \text{over } \mathbb{Q}. \tag{14}$$

When consistent, group the coefficients belonging to each subfield. Their linear combination is one summand in that subfield. Zero summands can be discarded. An exact basic solution has at most $N$ nonzero coordinate coefficients, so a witness with at most $N$ nonzero summands is enough. There is no need to enumerate all subsets of fields or all possible arities.

To minimize, sweep $d$ upward from $P^+(\deg_{\mathbb{Q}}(a))$ to $\deg_{\mathbb{Q}}(a)$ and stop at the first consistent system. With exhaustive subfields in a Galois ambient field, every smaller bound has been ruled out globally.

### 5.2 Why starting in the root field is useful but not always complete

The smaller field $\mathbb{Q}(a)$ often contains the desired summands, as the recovery formulas demonstrate for $s$. Searching there is usually much cheaper. But unless this field is Galois, the trace-descent theorem does not justify restricting all competitors to it. A positive witness is still valid. If its degree reaches an independently proved lower bound, global optimality follows anyway.

This is why the package defaults to the root field rather than automatically constructing an expensive normal closure. For a full global sum decision on another input, request `"NormalClosure" -> True` and check that `"SubfieldEnumerationComplete"` is true.

## 6 Enumerating embedded subfields with exact arithmetic

A subfield description must preserve its embedding in $K$. Two isomorphic abstract fields can occur as different subfields, with different coordinate subspaces and different effects on the target. Deduplicating only by minimal polynomials or isomorphism classes is therefore incorrect.

Here is a finite, self-contained enumeration method suitable for a reference implementation. It uses factorization over the ambient number field rather than requiring a separate Galois-group package. Exact extension factorization is provided by `FactorList[..., Extension -> theta]` [11].

Let $K = \mathbb{Q}(\theta)$ have degree $N$, with minimal polynomial $A(X)$. Factor it over $K$ as

$$A(X) = (X - \theta)q_1(X) \cdots q_s(X).$$

For any intermediate field $F \subseteq K$, the minimal polynomial of $\theta$ over $F$ is a product of some of these factors, necessarily including $X - \theta$.

**Proposition 6.1** (The coefficients recover the subfield)**.** *Let $h_F(X)$ be the minimal polynomial of $\theta$ over $F$. Then its coefficients generate $F$ over $\mathbb{Q}$.*

*Proof.* Let $F_0$ be their generated field. Clearly $F_0 \subseteq F$. The element $\theta$ satisfies $h_F$ over $F_0$, so

$$[K : F_0] = [F_0(\theta) : F_0] \leq \deg h_F = [K : F].$$

The reverse inequality follows from $F_0 \subseteq F$. Hence $F_0 = F$. □

To enumerate subfields of degree $m \mid N$, enumerate factor subsets for which

$$h(X) = (X - \theta) \prod_{j \in J} q_j(X) \quad \text{has degree } N/m.$$

Generate the field $F_h$ from the coefficients of $h$, and retain it exactly when $[F_h : \mathbb{Q}] = m$. Every subfield is found by Proposition 6.1. Conversely, each retained candidate is an actual embedded field of the claimed degree. The rational field and the ambient field can be inserted directly.

### 6.1 Generating a coefficient field without another primitive element

The implementation represents elements by rational coordinate vectors in a fixed power basis of $K$. Starting from the span of 1, repeatedly multiply current basis elements by the proposed generators and row-reduce the enlarged span. The dimension increases at most $N - 1$ times. At stabilization the span is a finite-dimensional $\mathbb{Q}$-subalgebra of a field, hence itself a field: multiplication by a nonzero element is an injective endomorphism, therefore surjective, so the inverse belongs to the subalgebra.

The reduced row-echelon basis is a canonical description of the *embedded rational subspace*. It is used for exact deduplication. Candidate generation stops early when the generated dimension already exceeds the desired field degree.

### 6.2 Completeness, normality, and cost

The enumeration method is finite but exponential in the number of irreducible factors of $A$ over $K$. In a normal closure all factors are linear; blindly enumerating subsets may be much more expensive than using an efficient subgroup lattice or a specialized subfield algorithm. The reference implementation uses a weighted depth-first subset traversal, pruning impossible degree sums and enforcing node and time budgets. It returns its partial fields together with an explicit completeness flag.

For the requested examples, the two cubic splitting fields have Galois groups $S_3$ and distinct quadratic subfields $\mathbb{Q}(\sqrt{-31})$, $\mathbb{Q}(\sqrt{-23})$. Their intersection is $\mathbb{Q}$: a nontrivial common Galois quotient of $S_3$ would force a common quadratic subfield. Their compositum therefore has degree 36. The degree-nine root fields are much smaller. Over either such root field, the target polynomial factors in degrees $1, 2, 2, 4$, corresponding to the orbits of the two independent conjugate transpositions. Only a small factor-subset search is needed to expose the two cubic fields. A full degree-36 normal closure is unnecessary to certify the examples' minima.

The proof of finiteness is not a promise of low running time. The complete theoretical mode removes the explicit budgets; the practical mode reports when it has not finished.

## 7 Two-factor products in fixed subfields are linear algebra

A direct coefficient ansatz

$$a = \left(\sum_i x_i f_i\right)\left(\sum_j y_j e_j\right)$$

looks bilinear. For *fixed subfields*, this is unnecessary.

**Theorem 7.1** (Subfield-intersection criterion)**.** *Let $F, E \subseteq K$ be subfields and let $a \in K^\times$. Then*

$$a \in F^\times E^\times \quad \Longleftrightarrow \quad F \cap aE \neq \{0\}.$$

*Proof.* If $a = bc$ with $b \in F^\times$ and $c \in E^\times$, then $b = a(1/c) \in F \cap aE$. Conversely, a nonzero element $b = aw$ of this intersection gives $a = b(1/w)$ with $w \in E^\times$. □

If the columns of $B_F, B_E$ are rational bases of the two fields in $K$, and $M_a$ is multiplication by $a$ in the ambient basis, compute

$$\ker\left[\, B_F \mid -M_a B_E \,\right]. \tag{15}$$

Any nonzero kernel vector produces $b = aw$. Neither its $F$-component nor its $E$-component can vanish, because each basis is linearly independent. This gives an exact witness without ideal factorization, fundamental units, integer relations among logarithms, or solving a rational-point problem in free coefficients.

The subfield property is crucial: an arbitrary vector space need not contain the reciprocal of a nonzero element. The argument is not a generic trick for linearizing arbitrary bilinear equations.

# 8 A complete two-factor criterion allowing external factors

One cannot simply assert that every product decomposition descends to the normal closure with the same bound. A safe reduction records the relative degree as a power.

**Theorem 8.1** (Power-and-subfield criterion)**.** *Let $K/\mathbb{Q}$ be finite Galois, let $a \in K^\times$, and let $d \geq 1$. The following are equivalent:*

1. *$a = bc$ for algebraic numbers $b, c$ with $\deg_{\mathbb{Q}}(b), \deg_{\mathbb{Q}}(c) \leq d$.*
2. *There are an integer $1 \leq t \leq d$ and subfields $F, E \subseteq K$ such that*

$$t[F : \mathbb{Q}] \leq d, \qquad t[E : \mathbb{Q}] \leq d, \qquad F \cap a^t E \neq \{0\}. \tag{16}$$

*Proof.* For necessity, suppose $a = bc$. Since $a \in K^\times$,

$$L = K(b) = K(c).$$

Put $t = [L : K]$, $F = K \cap \mathbb{Q}(b)$, and $E = K \cap \mathbb{Q}(c)$. Lemma 5.1 gives

$$\deg_{\mathbb{Q}}(b) = t[F : \mathbb{Q}], \qquad \deg_{\mathbb{Q}}(c) = t[E : \mathbb{Q}].$$

The norms

$$B = \mathrm{N}_{L/K}(b) \in F^\times, \qquad C = \mathrm{N}_{L/K}(c) \in E^\times$$

belong to the indicated fields because they are signed constant coefficients of the corresponding minimal polynomials. Multiplicativity of norm gives $a^t = BC$. Theorem 7.1 now gives the required intersection.

For sufficiency, choose $B \in F^\times$ and $w \in E^\times$ with $B = a^t w$. Take any complex $t$th root $b$ of $B$, and put $c = a/b$. Then

$$b^t = B, \qquad c^t = 1/w.$$

Consequently

$$\deg_{\mathbb{Q}}(b) \leq t[F : \mathbb{Q}] \leq d, \qquad \deg_{\mathbb{Q}}(c) \leq t[E : \mathbb{Q}] \leq d.$$

The product identity holds by the definition of $c$. No independent choice of its root branch is needed. □

This theorem yields a finite complete algorithm: for $d$ increasing, loop over $t = 1, \ldots, d$ and over all embedded subfields of degrees at most $\lfloor d/t \rfloor$, and perform the linear intersection test (15) with $a^t$ in place of $a$. Exhaustive negative results at smaller $d$ prove the two-factor minimum. The trivial pair $(a, 1)$ ensures success by $d = \deg_{\mathbb{Q}}(a)$.

A useful pruning condition is

$$\deg_{\mathbb{Q}}(a) \leq t[FE : \mathbb{Q}] \leq t[F : \mathbb{Q}][E : \mathbb{Q}],$$

since $a$ satisfies $X^t - a^t$ over $FE$. The theorem also accommodates asymmetric bounds $m, n$: require $t[F : \mathbb{Q}] \leq m$ and $t[E : \mathbb{Q}] \leq n$, with $t \leq \min(m, n)$.

## 8.1 Exact branch reconstruction in Wolfram Language

The intersection routine returns a nonzero $B \in F \cap a^t E$. The reconstruction is

```
beta  = RootReduce[B^(1/t)];
gamma = RootReduce[alpha/beta];
```

The principal complex power is an acceptable choice for `beta`; the quotient fixes the other branch consistently. The package then recomputes both absolute minimal-polynomial degrees and checks the product exactly. It never counts the relative exponent $t$ as the absolute degree of a factor.

### 8.2 Why a power must not be silently discarded

The product-descent proof in Altamirano's Theorem 3.1 defines a factor using a ratio $a_0/a_1$ of coefficients of a relative minimal polynomial [2]. The displayed argument does not establish $a_1 \neq 0$. Relative minimal polynomials can have vanishing linear coefficient, for example binomials of degree greater than one. This is a gap in that argument, not a claimed counterexample to its theorem. Theorem 8.1 avoids the division: the nonzero constant coefficient supplies a norm, and the relative degree is retained explicitly as $t$.

A concrete instance of the missing nonvanishing condition is

$$a = \sqrt{\sqrt{2}(1+\sqrt{3})}, \qquad b = 2^{1/4}, \qquad c = \sqrt{1+\sqrt{3}}, \qquad a = bc.$$

Their minimal polynomials have degrees eight, four, and four:

$$X^8 - 16X^4 + 16, \qquad X^4 - 2, \qquad X^4 - 2X^2 - 2.$$

The normal closure of $\mathbb{Q}(a)$ is $K = \mathbb{Q}(a, i)$: the eight roots are $\pm a, \pm ia, \pm 2/a, \pm 2i/a$. Put $F = \mathbb{Q}(\sqrt{2}, \sqrt{3})$; then $\mathbb{Q}(a) = F(a)$ is quadratic over $F$. The element $\sqrt{2}$ is not a square in $F$, since otherwise the non-Galois quartic field $\mathbb{Q}(2^{1/4})$ would equal the Galois field $F$. Also $1 + \sqrt{3}$ is not a square in $F$: writing a proposed square root as $x + y\sqrt{2}$ with $x, y \in \mathbb{Q}(\sqrt{3})$ forces $xy = 0$, whereas both $1 + \sqrt{3}$ and $(1 + \sqrt{3})/2$ have negative norms to $\mathbb{Q}$ and cannot be squares in $\mathbb{Q}(\sqrt{3})$. The quadratic-extension square test now shows that $b \notin F(a)$. Because $F(a)$ is real, $K \cap \mathbb{R} = F(a)$, so $b \notin K$ either. Nevertheless $b^2 = \sqrt{2} \in K$. Its relative minimal polynomial is therefore $X^2 - \sqrt{2}$, whose linear coefficient is zero. This example demonstrates the proof gap under the stated strict degree inequalities; it does not assert that the claimed descent conclusion is false.

The broader distinction between descent into a field and descent into radicals over that field also appears in norm-based work on metric Mahler measure [3]. Here the power is not treated as an inconvenience to be ignored; it is part of the exact decision criterion and of the degree accounting.

## 9 Why arbitrary-arity products are genuinely different

Consider

$$w = (1+\sqrt{2})(1+\sqrt{3})(1+\sqrt{6}) = 7 + 4\sqrt{2} + 3\sqrt{3} + 2\sqrt{6}. \tag{17}$$

Its minimal polynomial is

$$W(Z) = Z^4 - 28Z^3 + 128Z^2 - 200Z + 100.$$

The displayed three-factor witness and Theorem 3.1 show that $D_\times(w) = 2$. In sharp contrast,

$$\boxed{D_{\times,2}(w) = 4, \qquad D_\times(w) = 2.} \tag{18}$$

### 9.1 Proof that no two factors of degree below four suffice

Let $K = \mathbb{Q}(\sqrt{2}, \sqrt{3})$, a Galois field of degree four. Its proper nonrational subfields are precisely

$$\mathbb{Q}(\sqrt{2}), \qquad \mathbb{Q}(\sqrt{3}), \qquad \mathbb{Q}(\sqrt{6}).$$

The coefficients of $\sqrt{2}, \sqrt{3}, \sqrt{6}$ in (17) are all nonzero. Every nontrivial sign automorphism changes $w$, so $\mathbb{Q}(w) = K$ and $\deg_{\mathbb{Q}}(w) = 4$.

Suppose two factors of degrees at most three existed. Apply Theorem 8.1 with $d = 3$. For $t = 2$ or $t = 3$, the field-degree inequalities force $F = E = \mathbb{Q}$, so $w^t \in \mathbb{Q}$. This would give $\deg_{\mathbb{Q}}(w) \leq t < 4$, a contradiction. Thus $t = 1$, and only pairs of quadratic subfields need consideration. Two elements from the same quadratic field cannot multiply to $w$.

For two distinct quadratic fields, write $w$ in the product basis. A product of one element from each field has a rank-one $2 \times 2$ coefficient matrix. For the three possible pairs the matrices are

$$\begin{array}{lll} (\mathbb{Q}(\sqrt{2}), \mathbb{Q}(\sqrt{3})) & (\mathbb{Q}(\sqrt{2}), \mathbb{Q}(\sqrt{6})) & (\mathbb{Q}(\sqrt{3}), \mathbb{Q}(\sqrt{6})) \\ \begin{pmatrix} 7 & 3 \\ 4 & 2 \end{pmatrix} & \begin{pmatrix} 7 & 2 \\ 4 & 3/2 \end{pmatrix} & \begin{pmatrix} 7 & 2 \\ 3 & 4/3 \end{pmatrix}. \end{array}$$

Their determinants are 2, $5/2$, and $10/3$, all nonzero. No pair works. This excludes *external* factors as well, because the power-and-subfield theorem was global. Finally, $(w, 1)$ attains degree four, proving (18).

### 9.2 The failure of degree-decreasing recursion

An algorithm that searches for $a = bc$ with both $\deg_{\mathbb{Q}}(b), \deg_{\mathbb{Q}}(c) < \deg_{\mathbb{Q}}(a)$ and then recursively splits the factors cannot even begin on $w$. Nonetheless the optimal flat factorization is already visible in (17). Intermediate products of optimal low-degree factors may have the full degree of the target. Requiring the degree to decrease at every internal node removes valid solutions.

For sums there is an analogous distinction between controlling arity and merely controlling operand degree, but the vector-space criterion bypasses the arity search entirely. Multiplication has no corresponding linear-span closure: replacing a multiplicative group by its rational vector-space span changes the problem.

### 9.3 Norms and logarithms do not close this gap automatically

If $a = \prod_i b_i$ and a common extension $M/K$ contains the operands, then

$$a^{[M:K]} = \prod_i \mathrm{N}_{M/K}(b_i).$$

This is a statement about a power of $a$. Membership of that power in a product of small-field multiplicative groups is not sufficient to assert membership of $a$. Already $(\sqrt{2})^2 \in \mathbb{Q}^\times$, whereas $\sqrt{2}$ is not a product of rationals. Taking roots changes degrees; root-of-unity and branch factors must also be accounted for.

Numerical integer relations between logarithms can suggest multiplicative identities, but absolute values lose phases, complex logarithms introduce branch multiples of $2\pi i$, and a finite numerical search cannot prove the absence of a relation. Such tools are candidate generators only; the final product must be verified exactly, and minimality still needs a separate argument.

## 10 Certified arbitrary-arity product searches

The implemented practical search uses a finite catalog $C$ of nonzero exact algebraic numbers and a maximum arity $r$. It enumerates nondecreasing tuples of up to $r - 1$ catalog factors. For each prefix $b_1, \ldots, b_k$, it computes the unrestricted exact residual

$$c = \frac{a}{b_1 \cdots b_k}.$$

If $\deg_{\mathbb{Q}}(c) \le d$ and all prefix factors also have degree at most $d$, it returns a certified witness. The residual need not belong to the catalog. Intermediate product degrees are unrestricted, which is essential for the counterexample.

For example:

```
w = (1 + Sqrt[2]) (1 + Sqrt[3]) (1 + Sqrt[6]);
SearchRootProducts[w,
  {1 + Sqrt[2], 1 + Sqrt[3], 1 + Sqrt[6]},
  "MaximumDegree" -> 2, "MaximumFactors" -> 3,
  "IncludeInverses" -> False]
```

A degree-two witness attains the universal lower bound and therefore proves the *global arbitrary-arity* minimum, even though discovery used a finite catalog. With `"MaximumFactors" -> 2`, the same catalog search reports only bounded failure. Separately, the exhaustive two-factor algorithm proves the stronger negative statement from Section 9.

The precise finite search class is: at most the specified number of factors, with all but the last factor drawn from the supplied catalog (optionally augmented by inverses). Repeated factors are permitted. Inverses preserve algebraic degree. Nondecreasing catalog indices remove permutation duplicates without forbidding repeated entries.

### 10.1 A general positive-witness enumeration

There is a straightforward exhaustive enumeration of positive witnesses. At stage $H$, enumerate all primitive integer minimal polynomials of degree at most a chosen bound $d$ and height at most $H$, all their exact roots, and factor tuples of length at most $H$. Every finite degree-$d$ decomposition appears at a finite stage because its operands have finite minimal-polynomial heights. The residual trick lets one omit one factor from the enumerated tuple.

This is a *semidecision procedure*: it eventually finds a witness when one exists. At a fixed $d$, absence of a witness does not provide a stopping condition. To improve an upper bound without getting stuck forever at one unsuccessful $d$, dovetail the finite boxes for all $d$ below the best known witness. The reported upper bound eventually attains the true optimum, because an optimal witness occurs in some finite box. The procedure need not recognize when that final improvement has happened.

This distinction is the precise boundary of the general arbitrary-arity implementation here. It is not obscured by calling a finite search "factorization" or by returning the best factorization found as though it had been proved minimal.

## 11 Implementation architecture and invariants

The complete package is in `code/RootDecomposition.wl` and is reproduced in Appendix B. It uses no external number-field system. The central components are deliberately separated: exact input normalization, witness certification, finite search, ambient-field construction, subfield enumeration, rational linear algebra, and optimality classification.

### 11.1 Fixed integral primitive elements

The coordinate representation of $K = \mathbb{Q}(\theta)$ is

$$a = a_0 + a_1\theta + \cdots + a_{N-1}\theta^{N-1}, \qquad a_i \in \mathbb{Q}.$$

The generator is first made integral. If the primitive integer minimal polynomial of $a$ has leading coefficient $A$, then $\theta = Aa$ is integral and generates the same field. Indeed, multiplying the polynomial equation by $A^{n-1}$ produces a monic integer equation for $Aa$.

This normalization is an implementation requirement, not merely a convenience. The documentation guarantees that `ToNumberField[a, theta]` uses `AlgebraicNumber[theta, ...]` when `theta` is integral; otherwise it may autoreduce the generator. The package checks the returned generator explicitly. For a normal closure, it expresses all conjugate roots in their smallest common extension with `ToNumberField[roots, All]`; the default automatic form need not use that smallest common field [9].

`AlgebraicNumberPolynomial` then converts the representation to a polynomial in a local indeterminate, whose rational coefficients form the coordinate vector [10]. Multiplication becomes polynomial multiplication modulo the minimal polynomial of $\theta$. Subfield membership, span construction, and intersection tests thereafter use exact rational matrices rather than repeated numerical root recognition.

## 11.2 Certificates are separate from discovery

A candidate witness is accepted only after its identity is reduced exactly and each operand's minimal-polynomial degree is recomputed. The returned maximum is the maximum of those actual degrees. A certificate contains the operands as a list, so automatic evaluation of a surrounding sum or product cannot obscure the decomposition.

The important result fields are:

| Field | Meaning |
|---|---|
| `"IdentityVerified"` | The proposed exact sum or product equals the input. |
| `"MaximumDegree"` | The measured maximum absolute degree of the operands. |
| `"UniversalLowerBound"` | The automatically proved prime-divisor bound. |
| `"GlobalMinimumProved"` | The arbitrary-arity minimum is proved, either by attaining that lower bound or by a complete Galois-field sum search. |
| `"TwoFactorMinimumProved"` | The product minimum is proved within the class of two factors. This is not interchangeable with the previous field. |
| `"SubfieldEnumerationComplete"` | The specified embedded-subfield enumeration was actually exhausted. |

For $w$, a complete binary run should report maximum degree four, `"TwoFactorMinimumProved" -> True`, and `"GlobalMinimumProved" -> False`. A successful three-quadratic catalog run should report maximum degree two and global optimality true.

## 11.3 Resource and control-flow handling

The height search has separate template and pair budgets. Subfield enumeration has a visited-node budget and a time budget, retaining fields found before a limit. The final degree sweep has its own time budget. A time limit therefore applies to stages rather than guaranteeing a fixed total wall-clock duration. Wolfram's `TimeConstrained` concerns kernel CPU time and can behave differently under caching or multithreading; `Infinity` removes its constraint [13].

Positive witnesses remain useful from partial subfield lists. Negative completeness assertions do not. The sum solver can still return an upper bound from a partial enumeration, but it marks global optimality only when justified independently. Nested searches use tagged `Catch`/`Throw`; this avoids relying on `Return` to escape more than the innermost enclosing loop, a documented control-flow pitfall [12].

## 11.4 Typical calls

For the examples, begin with bounded-height search or the smaller root-field structural search:

```
MinimumRootSum[alphaS, "TimeConstraint" -> 120]
MinimumRootProductPair[alphaP, "TimeConstraint" -> 120]
```

For a genuinely exhaustive global sum search on a different input:

```
MinimumRootSum[alpha,
  "NormalClosure" -> True,
  "MaximumAmbientDegree" -> Infinity,
  "MaxNodes" -> Infinity,
  "TimeConstraint" -> Infinity]
```

The analogous options on `MinimumRootProductPair` give a complete *two-factor* minimizer. Neither call should be described as inexpensive for a large normal closure. In a larger application, a specialized embedded-subfield backend can replace the reference enumerator while preserving the same coordinate and certificate interfaces.

## 12 Verification and reproducibility

The archive includes two separate verification layers, whose status must not be conflated.

**Executed independent exact checks.** The script `verification/verify_exact.py` ran successfully with Python 3.13.5 and SymPy 1.14.0. All 40 checks passed. It verifies both resultants; irreducibility and unique-real-root isolation for the two cubics and degree-nine polynomials; the rational and polynomial recovery formulas; degree-nine power-basis recovery; bounded-height cubic discovery; the three-quadratic product identity; the quartic minimal polynomial; all subfield-intersection obstructions for two-factor bounds two and three; and the three absolute degrees in the vanishing-coefficient example below. The final check is only a lightweight delimiter/string/comment integrity check for the Wolfram package. SymPy's exact polynomial and number-field facilities are documented by the project [14].

For an additional algebraic audit, let the basis of the cubic compositum be

$$1, v, v^2, u, uv, uv^2, u^2, u^2v, u^2v^2.$$

The matrices whose columns are the coordinates of $1, a, \dots, a^8$ have determinants

$$\det M_{u+v} = -1\,339\,712, \qquad \det M_{uv} = -8.$$

They are nonsingular, proving that both targets generate the full cubic compositum. These rational computations do not depend on floating-point root ordering.

**Supplied native tests, not executed here.** The available Wolfram connector returned HTTP 404. The 23 tests in `code/Tests.wlt` are therefore provided for execution in Mathematica, not reported as passed. They exercise the actual public API, exact branch checks, rational normalization, inconsistent-system rejection, automatic discovery, biquadratic subfield completeness, resource-limit labeling, and the crucial distinction between binary and arbitrary-arity optimality.

Run the native suite with

```
TestReport["/absolute/path/to/code/Tests.wlt"]
```

and the independent verifier with

```
python verification/verify_exact.py
```

The latter regenerates readable and machine-readable reports. Independent algebraic checks validate the mathematics and the tested computational identities; they do not replace native evaluation tests for Wolfram-specific semantics or performance.

## 13 Practical conclusions

The two degree-nine inputs have exactly the decompositions requested, and their maximum degree three is globally optimal. The shortest exact solution for these specific inputs is given by the rational recovery formulas, followed by `RootReduce`; the bounded-height search discovers the cubics without supplying them in advance.

For general inputs, start with the absolute minimal polynomial and a rigorous lower bound. Keep candidate generation separate from exact verification. Search the root field first when that is economical, and enlarge to a normal closure only when the stronger completeness guarantee is needed. For sums, collect all eligible embedded subfields and solve one rational linear system at each degree bound. For two-factor products, use intersections $F \cap a^t E$ with the norm-derived degree constraints.

For arbitrary-arity products, retain high-degree intermediate products and use an explicitly bounded search or an explicitly nonterminating witness enumeration. The biquadratic counterexample proves that

recursively applying a binary minimizer is not a general solution. A returned decomposition becomes a global minimizer only when its degree meets a proved lower bound or another complete argument rules out every smaller bound. This separation gives useful exact answers without confusing successful simplification, bounded search, and mathematical optimality.

# A Copyable recovery functions for the two examples

These functions implement the identities of Section 2. They are specialized to the displayed cubics and target polynomials, not advertised as universal recognizers.

```
Clear[RecoverProductExample, RecoverSumExample];

RecoverProductExample[p_] := Module[{delta, a},
  delta = p^3 + p - 1;
  a = -(delta p + 1)/(delta^2 + p + 1);
  RootReduce /@ {a, a - delta}
];

RecoverSumExample[s_] := Module[{a},
  a = (3 s^5 - 5 s^3 - 3 s^2 + 2 s - 4)/
      (6 s^4 - 6 s + 4);
  RootReduce /@ {a, s - a}
];

RecoverProductExample[alphaP]
RecoverSumExample[alphaS]
(* Both lists represent exactly {u, v}. *)
```

# B Complete Wolfram Language reference package

The listing below is included directly from the accompanying package source, so the PDF and the distributed implementation stay synchronized. It is reference code with explicit completeness and verification status as described above. The examples and native regression tests are separate files in the archive.

Listing 1: `code/RootDecomposition.wl`

```
(* ::Package:: *)
(* Exact algebraic-number decomposition over Q.
   Native Wolfram regression tests are supplied separately in Tests.wlt.
   This implementation was not executed in a native Wolfram kernel by the author.
   Resource-limited or restricted searches never assert unrestricted nonexistence. *)

BeginPackage["RootDecomposition`"];
RootDegree::usage = "RootDegree[a] gives the absolute minimal-polynomial degree over Q.";
DegreeLowerBound::usage = "DegreeLowerBound[a] gives the largest prime divisor of RootDegree[a], or 1.";
ExactRootEqualQ::usage = "ExactRootEqualQ[a,b] tests equality by exact RootReduce.";
CertifyRootDecomposition::usage = "CertifyRootDecomposition[a,terms,op] certifies a flat Sum or Product witness.";
RootCompositionPolynomial::usage = "RootCompositionPolynomial[f,g,x,z,op] forms the sum/product resultant of monic
    normalizations.";
FindRootPairByHeight::usage = "FindRootPairByHeight[a,op] searches bounded-height rational polynomials and certifies
    root branches.";
BuildRootField::usage = "BuildRootField[a] constructs exact rational coordinates in Q(a), or in its normal closure.";
EnumerateEmbeddedSubfields::usage = "EnumerateEmbeddedSubfields[field] enumerates embedded subfields up to an
    explicit degree bound; reports completeness.";
MinimumRootSum::usage = "MinimumRootSum[a] searches all sums in the computed subfields. With a Galois ambient field
    and complete enumeration, its minimum is global.";
MinimumRootProductPair::usage = "MinimumRootProductPair[a] uses powers and subfield intersections. Completeness is
    for TWO factors, not arbitrary arity.";
```

```
SearchRootProducts::usage = "SearchRootProducts[a,catalog] searches bounded arity, with all but the last factor from
      catalog and an unrestricted exact residual.";

Begin["`Private`"];
$failureTag = Unique["RootDecompositionFailure"];
SetAttributes[protect, HoldAll];
protect[e_] := Catch[e, $failureTag];
fail[tag_, data_: <||>] := Throw[Failure[tag, data], $failureTag];
rationalQ[a_] := IntegerQ[a] || Head[a] === Rational;
zeroVectorQ[v_List] := And @@ (TrueQ[# === 0] & /@ v);
nonzeroRows[m_List] := Select[m, ! zeroVectorQ[#] &];
validOperation[op_] := If[! MemberQ[{"Sum", "Product"}, op],
  fail["Operation", <|"Expected" -> {"Sum", "Product"}|>]];
canonical[a_] := Module[{r},
  If[! FreeQ[a, _Real], fail["InexactInput", <|"Input" -> a|>]];
  r = Quiet[Check[RootReduce[a], $Failed]];
  If[r === $Failed, fail["RootReduceFailed", <|"Input" -> a|>]]; r
];
algDegree[a_] := Module[{r = canonical[a], x = Unique["x"], p, c},
  If[rationalQ[r], Return[1]];
  p = Quiet[Check[MinimalPolynomial[r, x], $Failed]];
  If[p === $Failed || ! PolynomialQ[p, x],
    fail["NotExactAlgebraic", <|"Input" -> a|>]];
  c = CoefficientList[p, x];
  If[! AllTrue[c, rationalQ] || Exponent[p, x] < 1,
    fail["NotExactAlgebraic", <|"Input" -> a|>]];
  Exponent[p, x]
];
lowerBound[a_] := Module[{n = algDegree[a]},
  If[n == 1, 1, Max[First /@ FactorInteger[n]]]
];
exactEqual[a_, b_] := TrueQ[canonical[a - b] === 0];
RootDegree[a_] := protect[algDegree[a]];
DegreeLowerBound[a_] := protect[lowerBound[a]];
ExactRootEqualQ[a_, b_] := protect[exactEqual[a, b]];

certificate[a_, terms_List, op_] := Module[{t, degrees, b, value, d},
  validOperation[op];
  If[terms === {}, fail["EmptyWitness"]];
  t = canonical /@ terms;
  value = If[op === "Sum", Total[t], Times @@ t];
  If[! exactEqual[a, value], fail["IdentityFailed", <|"Terms" -> t|>]];
  degrees = algDegree /@ t; d = Max[degrees]; b = lowerBound[a];
  <|"Status" -> "CertifiedWitness", "Operation" -> op,
    "Terms" -> t, "Degrees" -> degrees, "MaximumDegree" -> d,
    "UniversalLowerBound" -> b, "IdentityVerified" -> True,
    "GlobalMinimumProved" -> TrueQ[d == b],
    "OptimalityReason" -> If[d == b, "PrimeDivisorBound", "NotEstablished"]|>
];
CertifyRootDecomposition[a_, terms_List, op_] := protect[certificate[a, terms, op]];

compositionPolynomial[f_, g_, x_Symbol, z_Symbol, op_] := Module[{ff, gg, m, n, h},
  validOperation[op];
  If[x === z || ! PolynomialQ[f, x] || ! PolynomialQ[g, x], fail["PolynomialInput"]];
  m = Exponent[f, x]; n = Exponent[g, x];
  If[Min[m, n] < 1, fail["PositivePolynomialDegreeRequired"]];
  ff = Expand[f/Coefficient[f, x, m]];
  gg = Expand[g/Coefficient[g, x, n]];
  h = If[op === "Sum", Expand[gg /. x -> z - x],
    Sum[Coefficient[gg, x, j] z^j x^(n - j), {j, 0, n}]];
  Expand[Resultant[ff, h, x]]
];
RootCompositionPolynomial[f_, g_, x_Symbol, z_Symbol, op_] :=
  protect[compositionPolynomial[f, g, x, z, op]];
rootList[p_, x_Symbol] := Module[{fn = Function @@ {x, p}},
  Table[Root[fn, j], {j, 1, Exponent[p, x]}]
];

(* Primitive integer polynomials of positive leading coefficient and height <= h.
```

```
   This includes NONMONIC polynomials: rational, nonintegral factors are allowed. *)
smallPolynomials[d_Integer, h_Integer, x_Symbol, maxTemplates_] := Module[
  {out = {}, c, p, fl, estimate},
  estimate = Sum[h (2 h + 1)^m, {m, 1, d}];
  If[estimate > maxTemplates,
    fail["TemplateBudget", <|"RawTemplateCount" -> estimate, "Limit" -> maxTemplates|>]];
  Do[
    Do[
      c = Append[tail, lead];
      If[Apply[GCD, c] == 1,
        p = c . (x^Range[0, m]);
        fl = Rest[FactorList[p]];
        If[Length[fl] == 1 && fl[[1, 2]] == 1 &&
          Exponent[fl[[1, 1]], x] == m, AppendTo[out, p]]
      ], {tail, Tuples[Range[-h, h], m]}, {lead, 1, h}
    ], {m, 1, d}];
  SortBy[out, {Exponent[#, x] &, Count[CoefficientList[#, x], _?(# != 0 &)] &,
    Total[Abs[CoefficientList[#, x]]] &, CoefficientList[#, x] &}]
];
Options[FindRootPairByHeight] = {"MaximumDegree" -> 3, "CoefficientHeight" -> 1,
  "MaxTemplates" -> 20000, "MaxPairs" -> 20000, "TimeConstraint" -> 60};
FindRootPairByHeight[a_, op_, OptionsPattern[]] := protect[TimeConstrained[
  Module[{aa = canonical[a], n, lb, maxd = OptionValue["MaximumDegree"],
    h = OptionValue["CoefficientHeight"], x = Unique["x"], z = Unique["z"],
    p, polys, degs, count = 0, r, rr1, rr2, ans, found = Unique["found"]},
    validOperation[op]; n = algDegree[aa]; lb = lowerBound[aa];
    If[n == 1, Return[certificate[aa, {aa}, op]]];
    If[! IntegerQ[maxd] || maxd < 1 || ! IntegerQ[h] || h < 1, fail["SearchBounds"]];
    polys = smallPolynomials[maxd, h, x, OptionValue["MaxTemplates"]];
    degs = Exponent[#, x] & /@ polys;
    p = MinimalPolynomial[aa, z]; p = p/Coefficient[p, z, n];
    Catch[
      Do[Do[
        If[Max[degs[[i]], degs[[j]]] == d && degs[[i]] degs[[j]] >= n,
          count++;
          If[count > OptionValue["MaxPairs"], fail["PairBudget", <|"Pairs" -> count - 1|>]];
          r = compositionPolynomial[polys[[i]], polys[[j]], x, z, op];
          If[TrueQ[PolynomialRemainder[r, p, z] === 0],
            rr1 = rootList[polys[[i]], x]; rr2 = rootList[polys[[j]], x];
            Do[
              If[exactEqual[aa, If[op === "Sum", b + c, b c]],
                ans = certificate[aa, {b, c}, op];
                Throw[Join[ans, <|"SearchScope" -> "TwoFactorsInPolynomialHeightBox",
                  "CoefficientHeight" -> h, "PairsTested" -> count,
                  "Polynomials" -> {polys[[i]], polys[[j]]}, "PolynomialVariable" -> x|>], found]
              ], {b, rr1}, {c, rr2}]
          ]
        ], {i, Length[polys]}, {j, i, Length[polys]}], {d, lb, maxd}];
      <|"Status" -> "NotFoundInBounds", "MaximumDegree" -> maxd,
        "CoefficientHeight" -> h, "PairsTested" -> count,
        "SearchScope" -> "TwoFactorsInPolynomialHeightBox", "GlobalNonexistenceProved" -> False|>,
      found]
  ], OptionValue["TimeConstraint"], Failure["TimeLimit", <|"SearchComplete" -> False|>]]];

(* Rational coordinate arithmetic in Q(theta). The generator theta is integral
   so ToNumberField retains it, rather than silently changing the power basis. *)
integralGenerator[a_] := Module[{r = canonical[a], x = Unique["x"], p, c, scale},
  If[rationalQ[r], Return[0]];
  p = MinimalPolynomial[r, x]; c = CoefficientList[p, x];
  scale = Apply[LCM, Denominator[c]];
  p = Expand[scale p]; c = CoefficientList[p, x];
  p = p/Apply[GCD, c];
  canonical[Coefficient[p, x, Exponent[p, x]] r]
];
coordinates[a_, ctx_Association] := Module[{r = canonical[a], n = ctx["Degree"], an, p, v},
  If[rationalQ[r], Return[PadRight[{r}, n]]];
  an = Quiet[Check[ToNumberField[r, ctx["Theta"]], $Failed]];
  If[Head[an] =!= AlgebraicNumber, fail["NotInAmbientField", <|"Element" -> r|>]];
  If[! exactEqual[an[[1]], ctx["Theta"]], fail["UnexpectedPrimitiveElement"]];
```

```
  p = AlgebraicNumberPolynomial[an, ctx["Variable"]];
  v = CoefficientList[p, ctx["Variable"]];
  If[Length[v] > n || ! AllTrue[v, rationalQ], fail["CoordinateConversionFailed"]];
  PadRight[v, n]
];
fromCoordinates[v_List, ctx_Association] := canonical[v .
  Prepend[Table[ctx["Theta"]^k, {k, 1, ctx["Degree"] - 1}], 1]];
mulCoordinates[v_List, w_List, ctx_Association] := Module[{x = ctx["Variable"], p},
  p = PolynomialRemainder[Expand[(v . ctx["Powers"])(w . ctx["Powers"])], ctx["Polynomial"], x];
  PadRight[CoefficientList[p, x], ctx["Degree"]]
];
powerCoordinates[v_List, k_Integer, ctx_Association] := Module[
  {r = PadRight[{1}, ctx["Degree"]], b = v, j = k},
  While[j > 0,
    If[OddQ[j], r = mulCoordinates[r, b, ctx]];
    j = Quotient[j, 2]; If[j > 0, b = mulCoordinates[b, b, ctx]]]; r
];
Options[BuildRootField] = {"NormalClosure" -> False, "MaximumAmbientDegree" -> 64,
  "TimeConstraint" -> 60};
BuildRootField[a_, OptionsPattern[]] := protect[TimeConstrained[
  Module[{aa = canonical[a], x = Unique["x"], p, n, theta, nf, gens, factors, ctx},
    n = algDegree[aa];
    If[n > OptionValue["MaximumAmbientDegree"], fail["AmbientDegreeBudget", <|"Degree" -> n|>]];
    If[TrueQ[OptionValue["NormalClosure"]] && n > 1,
      p = MinimalPolynomial[aa, x];
      nf = Quiet[Check[ToNumberField[rootList[p, x], All], $Failed]];
      If[! ListQ[nf], fail["NormalClosureConstructionFailed"]];
      gens = Cases[nf, AlgebraicNumber[t_, _] :> t, {1}];
      If[gens === {}, fail["PrimitiveElementMissing"]];
      theta = integralGenerator[First[gens]],
      theta = integralGenerator[aa]
    ];
    n = algDegree[theta];
    If[n > OptionValue["MaximumAmbientDegree"], fail["AmbientDegreeBudget", <|"Degree" -> n|>]];
    p = MinimalPolynomial[theta, x]; p = Expand[p/Coefficient[p, x, n]];
    factors = If[n == 1, {p},
      Quiet[Check[First /@ Rest[FactorList[p, Extension -> theta]], $Failed]]];
    If[! ListQ[factors] || factors === {}, fail["AmbientFactorizationFailed"]];
    factors = Expand[#/Coefficient[#, x, Exponent[#, x]]] & /@ factors;
    ctx = <|"Alpha" -> aa, "Theta" -> theta, "Variable" -> x,
      "Polynomial" -> p, "Powers" -> x^Range[0, n - 1], "Degree" -> n,
      "Factors" -> factors, "Galois" -> AllTrue[factors, Exponent[#, x] == 1 &]|>;
    If[TrueQ[OptionValue["NormalClosure"]] && ! TrueQ[ctx["Galois"]], fail["NormalityCheckFailed"]];
    Join[ctx, <|"AlphaCoordinates" -> coordinates[aa, ctx]|>]
  ], OptionValue["TimeConstraint"], Failure["TimeLimit", <|"Stage" -> "AmbientField"|>]]];

(* The Q-subalgebra generated by coefficient vectors is a field because it is
   a finite-dimensional domain. Row-reduced bases distinguish EMBEDDED fields. *)
generatedFieldRows[gens_List, ctx_Association, cap_: Infinity] := Module[
  {basis = {PadRight[{1}, ctx["Degree"]]}, products, next, changed = True},
  While[changed && Length[basis] <= cap,
    products = Flatten[Table[mulCoordinates[b, g, ctx], {b, basis}, {g, gens}], 1];
    next = nonzeroRows[RowReduce[Join[basis, products]]];
    changed = Length[next] != Length[basis]; basis = next
  ];
  If[Length[basis] > cap, $Failed, basis]
];
Options[EnumerateEmbeddedSubfields] = {"MaximumDegree" -> Automatic,
  "MaxNodes" -> 100000, "TimeConstraint" -> 60};
EnumerateEmbeddedSubfields[ctx_Association, OptionsPattern[]] := protect[Module[
  {n = ctx["Degree"], x = ctx["Variable"], maxd = OptionValue["MaximumDegree"],
    fields = {}, add, f = ctx["Factors"], pos, others, weights, tailWeights,
    q0, visit, process, nodes = 0, candidates = 0, stop, budget = Unique["budget"],
    maxnodes = OptionValue["MaxNodes"], ad = algDegree[ctx["Alpha"]]},
  If[maxd === Automatic, maxd = ad];
  If[! IntegerQ[maxd] || maxd < 1, fail["SubfieldDegreeBound"]];
  maxd = Min[maxd, n];
  add[rows_List] := If[! AnyTrue[fields, #["Rows"] === rows &],
    AppendTo[fields, <|"Degree" -> Length[rows], "Rows" -> rows|>]];
```

```
  add[{PadRight[{1}, n]}];
  If[n <= maxd, add[IdentityMatrix[n]]];
  If[ad <= maxd, add[nonzeroRows[RowReduce[
    Table[powerCoordinates[ctx["AlphaCoordinates"], k, ctx], {k, 0, ad - 1}]]]]];
  pos = Select[Range[Length[f]], Exponent[f[[#]], x] == 1 &&
      exactEqual[f[[#]] /. x -> ctx["Theta"], 0] &];
  If[Length[pos] != 1, fail["DistinguishedLinearFactorMissing"]];
  others = Delete[f, {First[pos]}]; weights = Exponent[#, x] & /@ others;
  tailWeights = Reverse[Accumulate[Reverse[weights]]];
  q0 = x - ctx["Theta"];
  process[chosen_List, m_Integer] := Module[{q, gens, rows},
    candidates++;
    q = Expand[q0 (Times @@ others[[chosen]])];
    gens = coordinates[#, ctx] & /@ CoefficientList[q, x];
    rows = generatedFieldRows[gens, ctx, m];
    If[rows =!= $Failed && Length[rows] == m, add[rows]]
  ];
  visit[i_Integer, remaining_Integer, chosen_List, m_Integer] := (
    nodes++;
    If[nodes > maxnodes, Throw["NodeBudget", budget]];
    If[remaining == 0, process[chosen, m],
      If[i <= Length[others] && 0 < remaining <= tailWeights[[i]],
        visit[i + 1, remaining, chosen, m];
        If[weights[[i]] <= remaining,
          visit[i + 1, remaining - weights[[i]], Append[chosen, i], m]]]]);
  stop = TimeConstrained[
    Catch[
      Do[visit[1, n/m - 1, {}, m],
        {m, Select[Divisors[n], 1 < # < n && # <= maxd &]}]; "Complete", budget],
    OptionValue["TimeConstraint"], "TimeLimit"];
  <|"Ambient" -> ctx, "Fields" -> SortBy[fields, #["Degree"] &],
    "Complete" -> (stop === "Complete"), "StopReason" -> stop,
    "MaximumDegree" -> maxd, "VisitedNodes" -> nodes,
    "CandidateProducts" -> candidates|>
]];

(* Solve M.c=b over Q; set free variables to zero. No numerical rank tests. *)
rationalSolve[m_List, b_List] := Module[{cols = Length[First[m]], rr, c, nz, pivot},
  rr = RowReduce[MapThread[Append, {m, b}]]; c = ConstantArray[0, cols];
  If[AnyTrue[rr, zeroVectorQ[Take[#, cols]] && Last[#] =!= 0 &], Return[$Failed]];
  Do[
    nz = Select[Range[cols], row[[#]] =!= 0 &];
    If[nz =!= {}, pivot = First[nz]; c[[pivot]] = row[[-1]]],
    {row, rr}]; c
];
sumFromFields[ctx_Association, fields_List, d_Integer] := Module[
  {selected, rows, coeffs, terms = {}, k = 0, len, v},
  selected = Select[fields, #["Degree"] <= d &];
  rows = Flatten[#["Rows"] & /@ selected, 1];
  coeffs = rationalSolve[Transpose[rows], ctx["AlphaCoordinates"]];
  If[coeffs === $Failed, Return[$Failed]];
  Do[
    len = field["Degree"];
    v = Take[coeffs, {k + 1, k + len}] . field["Rows"]; k += len;
    If[! zeroVectorQ[v], AppendTo[terms, fromCoordinates[v, ctx]]],
    {field, selected}];
  If[terms === {}, {0}, terms]
];
pairFromFields[ctx_Association, fields_List, d_Integer] := Module[
  {selected, aPower, frows, erows, ns, v, b, beta, gamma, found = Unique["pair"]},
  Catch[
    Do[
      selected = Select[fields, t #["Degree"] <= d &];
      aPower = powerCoordinates[ctx["AlphaCoordinates"], t, ctx];
      Do[
        frows = selected[[i]]["Rows"]; erows = selected[[j]]["Rows"];
        ns = NullSpace[Transpose[Join[frows, -mulCoordinates[aPower, #, ctx] & /@ erows]]];
        If[ns =!= {},
          v = Take[First[ns], Length[frows]] . frows;
```

```
          If[zeroVectorQ[v], fail["UnexpectedZeroIntersectionVector"]];
          b = fromCoordinates[v, ctx];
          beta = canonical[b^(1/t)]; gamma = canonical[ctx["Alpha"]/beta];
          If[Max[algDegree[beta], algDegree[gamma]] > d, fail["DegreeCertificateFailed"]];
          Throw[<|"Terms" -> {beta, gamma}, "Power" -> t,
            "SubfieldDegrees" -> {Length[frows], Length[erows]}|>, found]
        ], {i, Length[selected]}, {j, i, Length[selected]}],
      {t, 1, d}]; $Failed, found]
];
Options[MinimumRootSum] = {"NormalClosure" -> False, "MaximumAmbientDegree" -> 64,
  "MaxNodes" -> 100000, "TimeConstraint" -> 60};
Options[MinimumRootProductPair] = Options[MinimumRootSum];

(* The time limit applies separately to field construction, subfield enumeration,
   and the final bound sweep. A partial enumeration remains usable for witnesses. *)
minimumInFields[a_, op_, normal_, maxAmbient_, maxNodes_, seconds_] := Module[
  {ctx, enumeration, fields, n, lb, witness, ans, globalScope, found = Unique["answer"]},
  n = algDegree[a]; lb = lowerBound[a];
  If[n == 1, Return[certificate[a, {canonical[a]}, op]]];
  ctx = BuildRootField[a, "NormalClosure" -> normal,
    "MaximumAmbientDegree" -> maxAmbient, "TimeConstraint" -> seconds];
  If[FailureQ[ctx], Return[ctx]];
  enumeration = EnumerateEmbeddedSubfields[ctx, "MaximumDegree" -> n,
    "MaxNodes" -> maxNodes, "TimeConstraint" -> seconds];
  If[FailureQ[enumeration], Return[enumeration]];
  fields = enumeration["Fields"];
  globalScope = TrueQ[ctx["Galois"] && enumeration["Complete"]];
  TimeConstrained[
    Catch[
      Do[
        witness = If[op === "Sum", sumFromFields[ctx, fields, d], pairFromFields[ctx, fields, d]];
        If[witness =!= $Failed,
          ans = certificate[a, If[op === "Sum", witness, witness["Terms"]], op];
          ans = Join[ans, <|"AmbientDegree" -> ctx["Degree"], "AmbientGalois" -> ctx["Galois"],
            "SubfieldEnumerationComplete" -> enumeration["Complete"],
            "SubfieldsFound" -> Length[fields], "ExcludedBoundsInSearchScope" -> Range[lb, d - 1]|>];
          If[op === "Sum",
            ans = Join[ans, <|"GlobalMinimumProved" -> (ans["GlobalMinimumProved"] || globalScope),
              "AmbientMinimumProved" -> enumeration["Complete"],
              "OptimalityReason" -> If[ans["GlobalMinimumProved"], "PrimeDivisorBound",
                If[globalScope, "TraceDescentAndExhaustiveSubfields", "RestrictedSearch"]]|>],
            ans = Join[ans, <|"TwoFactorMinimumProved" -> (ans["GlobalMinimumProved"] || globalScope),
              "Power" -> witness["Power"], "SubfieldDegrees" -> witness["SubfieldDegrees"],
              "SearchScope" -> If[globalScope, "AllAlgebraicTwoFactorDecompositions", "ComputedPowerSubfieldFamily"
    ]|>]
          ];
          Throw[ans, found]
        ], {d, lb, n}];
      fail["InternalNoTrivialWitness"], found],
    seconds, Failure["TimeLimit", <|"Stage" -> "DegreeSweep", "SearchComplete" -> False|>]]
];
MinimumRootSum[a_, OptionsPattern[]] := protect[minimumInFields[a, "Sum",
  OptionValue["NormalClosure"], OptionValue["MaximumAmbientDegree"],
  OptionValue["MaxNodes"], OptionValue["TimeConstraint"]]];
MinimumRootProductPair[a_, OptionsPattern[]] := protect[minimumInFields[a, "Product",
  OptionValue["NormalClosure"], OptionValue["MaximumAmbientDegree"],
  OptionValue["MaxNodes"], OptionValue["TimeConstraint"]]];

Options[SearchRootProducts] = {"MaximumDegree" -> 3, "MaximumFactors" -> 3,
  "IncludeInverses" -> True, "MaxNodes" -> 100000, "TimeConstraint" -> 60};
SearchRootProducts[a_, catalog_List, OptionsPattern[]] := protect[TimeConstrained[
  Module[{aa = canonical[a], lb, maxd = OptionValue["MaximumDegree"],
    maxk = OptionValue["MaximumFactors"], maxnodes = OptionValue["MaxNodes"],
    cats, active, nodes = 0, visit, ans, found = Unique["catalogWitness"], d},
    If[! IntegerQ[maxd] || maxd < 1 || ! IntegerQ[maxk] || maxk < 1, fail["SearchBounds"]];
    lb = lowerBound[aa];
    If[algDegree[aa] == 1, Return[certificate[aa, {aa}, "Product"]]];
    cats = canonical /@ catalog;
    cats = Select[cats, ! exactEqual[#, 0] && ! exactEqual[#, 1] &];
```

```
    If[TrueQ[OptionValue["IncludeInverses"]], cats = Join[cats, canonical[1/#] & /@ cats]];
    cats = DeleteDuplicates[cats, exactEqual];
    visit[start_Integer, remaining_Integer, prefix_List, product_] := Module[{residual},
      nodes++;
      If[nodes > maxnodes, fail["NodeBudget", <|"VisitedNodes" -> nodes - 1|>]];
      If[remaining == 0,
        residual = canonical[aa/product];
        If[algDegree[residual] <= d,
          ans = certificate[aa, Append[prefix, residual], "Product"];
          Throw[Join[ans, <|"SearchScope" -> "AllButLastFactorFromCatalog",
            "MaximumFactors" -> maxk, "NodesTested" -> nodes|>], found]],
        Do[visit[j, remaining - 1, Append[prefix, active[[j]]], canonical[product active[[j]]],
          {j, start, Length[active]}]]
    ];
    Catch[
      Do[active = Select[cats, algDegree[#] <= d &];
        Do[visit[1, k - 1, {}, 1], {k, 1, maxk}], {d, lb, maxd}];
      <|"Status" -> "NotFoundInBounds", "SearchScope" -> "AllButLastFactorFromCatalog",
        "MaximumDegree" -> maxd, "MaximumFactors" -> maxk, "NodesTested" -> nodes,
        "GlobalNonexistenceProved" -> False|>, found]
  ], OptionValue["TimeConstraint"], Failure["TimeLimit", <|"SearchComplete" -> False|>]];

End[];
EndPackage[];
```

## References

[1] Vladimir Reshetnikov, “Factor a polynomial root into roots of smallest possible degree,” Mathematica Stack Exchange, question 105933, with answers by yarchik and Daniel Lichtblau. https://mathematica.stackexchange.com/questions/105933/factor-a-polynomial-root-into-roots-of-smallest-possible-degree.

[2] Christian Altamirano, *A Problem in Algebraic Number Theory*, UROP+ final paper, 31 August 2017. Mentor: Atticus Christensen; project suggested by Bjorn Poonen. https://math.mit.edu/research/undergraduate/urop-plus/documents/2017/Altamirano.pdf.

[3] Charles L. Samuels, *The infimum in the metric Mahler measure*, arXiv:1408.4165, 2014. https://arxiv.org/abs/1408.4165.

[4] J. S. Milne, *Fields and Galois Theory*, version 5.10, September 2022. https://www.jmilne.org/math/CourseNotes/FT.pdf.

[5] Wolfram Research, *Root*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Root.html.

[6] Wolfram Research, *MinimalPolynomial*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/MinimalPolynomial.html.

[7] Wolfram Research, *RootReduce*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/RootReduce.html.

[8] Wolfram Research, *Resultant*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Resultant.html.

[9] Wolfram Research, *ToNumberField*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/ToNumberField.html.

[10] Wolfram Research, *AlgebraicNumberPolynomial*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/AlgebraicNumberPolynomial.html.

[11] Wolfram Research, *FactorList*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/FactorList.html.

[12] Wolfram Research, *Return*, especially “Possible Issues,” Wolfram Language documentation. https://reference.wolfram.com/language/ref/Return.html.

[13] Wolfram Research, *TimeConstrained*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/TimeConstrained.html.

[14] SymPy Development Team, *Number Fields*, SymPy documentation. https://docs.sympy.org/latest/modules/polys/numberfields.html.

Online documentation and linked materials consulted 6 September 2026. The downloadable package includes the article source, PDF, implementation, examples, native tests, and independent verification records; it does not redistribute the cited papers.